PedeJá

Documentação de Arquitetura de Software (arc42)

Arquitetura de Software em Nuvem — TDE 2 — Documentação de Arquitetura

Integrantes do grupo

Rodrigo Petrelli

Luiz Eduardo Assi

arc42 · C4 Model (níveis 1 a 4) · UML · DER · Software Architecture Canvas

Versão 1.0 · 15 de outubro de 2026

Sobre o arc42

arc42, o template para documentação de software e arquitetura de sistemas.

Versão do template 8.2-PT. (based upon AsciiDoc version), Setembro de 2024

Criado, mantido e © pelo Dr. Peter Hruschka, Dr. Gernot Starke e colaboradores. Veja https://arc42.org.

Sumário

1. Introdução e Objetivos 5
1.1 Visão Geral dos Requisitos 5
1.2 Objetivos de Qualidade 5
1.3 Partes Interessadas 6
2. Restrições Arquiteturais 7
3. Contexto e Escopo 8
3.1 Contexto Negocial (C4 Nível 1) 8
3.2 Contexto Técnico 8
4. Estratégia de Solução 10
4.1 Software Architecture Canvas 11
5. Visão de Blocos de Construção 12
5.1 Visão Sistêmica Geral de Caixa Branca (C4 Nível 2 — Containers) 12
5.2 Nível 2 (C4 Nível 3 — Componentes) 13
5.2.1 Caixa Branca Serviço Pedidos 14
5.2.2 Caixa Branca API Gateway e BFFs 15
5.2.3 Caixa Branca Serviço Notificações (Serverless) 16
5.2.4 Caixa Branca Web Shell (Microfrontends) 16
5.2.5 Estrutura do código: Clean Architecture × Vertical Slice 17
5.3 Nível 3 (C4 Nível 4 — Código em UML) 18
5.3.1 Caixa Branca Pedidos: classes de casos de uso, portas e adaptadores 19
5.3.2 Caixa Branca Pedidos: classes do domínio 20
5.3.3 Caixa Branca Shared Kernel: classes de erros, portas e eventos 21
5.3.4 Caixa Branca Pedidos: componentes UML 22
6. Visão de Tempo de Execução 23
6.1 Criar pedido e notificar o restaurante 24
6.2 Cadastro de produto e replicação para Pedidos (EDA) 26
6.3 Vitrine do restaurante via BFF Web 27

6.4 Confirmar / cancelar pedido 28
6.5 Ciclo de vida do pedido 29
7. Visão de Implantação 30
7.1 Nível de Infraestrutura 1 (AWS — produção) 30
7.2 Nível de Infraestrutura 2 31
7.2.1 Ambiente local de desenvolvimento 31
8. Conceitos Transversais 32
8.1 Modelo de dados e Database per Service (DER) 32
8.2 Arquitetura orientada a eventos 33
8.3 Regras de Clean Architecture e Vertical Slice 33
8.4 Testes unitários de arquitetura 33
8.5 Tratamento de erros 34
8.6 Segurança, observabilidade e configuração 35
9. Decisões Arquiteturais 36
10. Requisitos de Qualidade 37
10.1 Árvore de Qualidade 37
10.2 Cenários de Qualidade 37
11. Riscos e Débitos Técnicos 38
12. Glossário 39

1. Introdução e Objetivos

O **PedeJá** é uma plataforma de pedidos de delivery de restaurantes. Clientes consultam cardápios e fazem pedidos; restaurantes parceiros mantêm o cardápio e confirmam ou cancelam pedidos; a plataforma notifica as partes a cada mudança. A versão de partida era um único arquivo de rotas Express, com validação, regras de negócio, acesso a dados e notificação dentro dos handlers. Este documento descreve a arquitetura-alvo e a refatoração entregue no repositório, que aplica os estilos arquiteturais pedidos: *Serverless, Microfrontends, BFF + Microsserviços + Database per Service, API Gateway, Arquitetura Orientada a Eventos, Clean Architecture com Vertical Slice e Testes Unitários de Arquitetura*.

1.1 Visão Geral dos Requisitos

ID	Requisito funcional	Endpoint	Serviço / fatia
RF01	Cadastrar restaurante (nome, categoria)	`POST /api/restaurantes`	Catálogo / criar-restaurante
RF02	Listar e consultar restaurantes	`GET /api/restaurantes[/:id]`	Catálogo / listar-restaurantes, obter-restaurante
RF03	Cadastrar produto no cardápio (preço inteiro > 0 em centavos)	`POST /api/restaurantes/:id/produtos`	Catálogo / cadastrar-produto
RF04	Listar cardápio do restaurante	`GET /api/restaurantes/:id/produtos`	Catálogo / listar-cardapio
RF05	Cadastrar cliente com e-mail válido e único	`POST /api/clientes`	Clientes / cadastrar-cliente
RF06	Criar pedido: itens do mesmo restaurante, disponíveis; total = subtotal + taxa de R$ 7,90	`POST /api/pedidos`	Pedidos / criar-pedido
RF07	Consultar pedido e histórico do cliente	`GET /api/pedidos/:id`, `GET /api/clientes/:id/pedidos`	Pedidos / consultar-pedido, listar-pedidos-cliente
RF08	Confirmar (CRIADO→CONFIRMADO) e cancelar (CRIADO|CONFIRMADO→CANCELADO)	`PATCH /api/pedidos/:id/confirmar|cancelar`	Pedidos / confirmar-pedido, cancelar-pedido
RF09	Notificar restaurante (novo pedido) e cliente (mudança de status)	— (eventos)	Notificações / notificar-*
RF10	Telas agregadas: vitrine do restaurante, histórico e resumo móvel	`/bff/web/*`, `/bff/mobile/*`	BFF Web, BFF Mobile

Erros seguem o contrato `{"erro": "mensagem"}`: 400 (validação), 404 (não encontrado), 422 (regra de negócio) e 429 (rate limit).

1.2 Objetivos de Qualidade

Prioridade	Objetivo	Motivação
1	**Manutenibilidade / evolutividade**	O código inicial misturava tudo nas rotas. Cada funcionalidade nova deve ser isolada em uma fatia, sem efeitos colaterais.
2	**Escalabilidade elástica**	Delivery tem picos fortes no almoço e no jantar. Os serviços devem escalar de forma independente e custar pouco na ociosidade.
3	**Disponibilidade / resiliência**	Falha em uma capacidade secundária, como notificações, não pode impedir a criação de pedidos.

Prioridade	Objetivo	Motivação
4	**Implantabilidade independente**	Times distintos (Catálogo, Pedidos, Front) devem publicar sem coordenar deploys.
5	**Testabilidade e conformidade arquitetural**	As regras de dependência precisam ser verificadas automaticamente, e não só documentadas.

1.3 Partes Interessadas

Função/Nome	Contato	Expectativas
Cliente final	Canais web e mobile	Fazer pedidos rapidamente, ver o total correto e acompanhar o status.
Restaurante parceiro	Painel web; notificações	Manter o cardápio e ser avisado de novos pedidos imediatamente.
Times de desenvolvimento (Catálogo, Clientes, Pedidos, Front)	Integrantes do grupo (capa)	Autonomia de código, dados e deploy; regras de arquitetura claras e verificáveis.
Operação / SRE	Integrantes do grupo (capa)	Observabilidade (correlation id), escalabilidade automática e baixo custo operacional.
Professor / avaliadores	Disciplina	Rastreabilidade entre os estilos arquiteturais, os diagramas e o código entregue.

2. Restrições Arquiteturais

Tipo	Restrição	Consequência
Técnica	TypeScript (strict) em Node.js ≥ 24; Express 4 como adaptador HTTP local.	Mesma linguagem no backend, BFFs e handlers Lambda.
Técnica	Nuvem-alvo AWS: API Gateway, Lambda, EventBridge, DynamoDB, S3 + CloudFront, SNS/SES.	Estilos Serverless e EDA usam serviços gerenciados.
Técnica	Valores monetários em centavos (inteiros).	Sem erros de ponto flutuante nos totais.
Técnica	Sem framework de injeção de dependência.	Injeção manual em composition roots (`*.service.ts`, `app.ts`, `lambda.ts`).
Organizacional	Trabalho em grupo de 4 integrantes; estilos arquiteturais definidos pelo enunciado.	Todos os estilos aparecem no código ou na arquitetura-alvo documentada.
Organizacional	Clean Architecture, Vertical Slice e testes de arquitetura devem refletir o código entregue.	Estrutura de pastas e testes em `backend/tests/arquitetura` fazem parte do build.
Convenção	Linguagem ubíqua em português (Pedido, Cardápio, Restaurante) no domínio e nos contratos.	Nomes de classes, eventos e endpoints em pt-BR.
Convenção	Fatia = pasta `features/<fatia>/` com `<fatia>.ts` (caso de uso) e `<fatia>.http.ts` (adaptador).	Convenção usada pelos testes de arquitetura para classificar arquivos.

3. Contexto e Escopo

3.1 Contexto Negocial (C4 Nível 1)

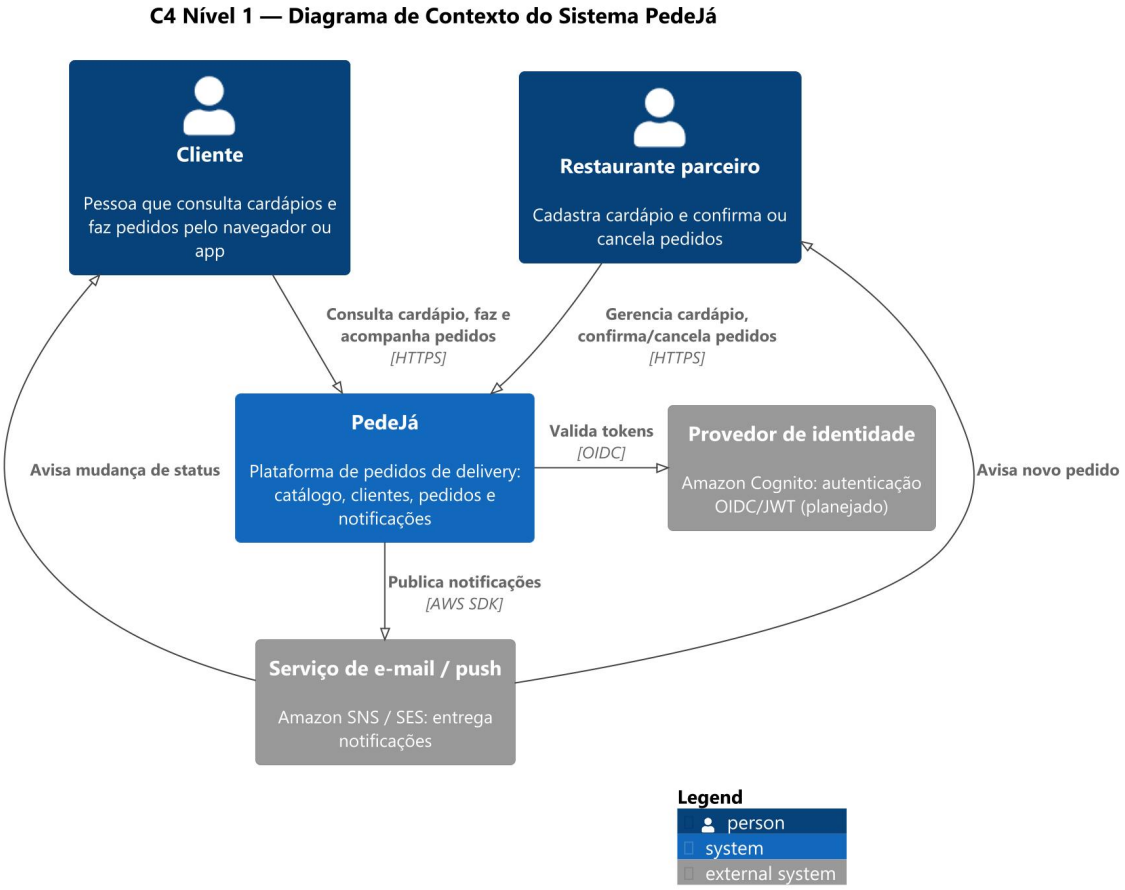


Figura 1 — C4 Nível 1: diagrama de contexto do sistema PedeJá

Parceiro	Entrada para o PedeJá	Saída do PedeJá
Cliente	Cadastro, pedidos (itens e quantidades), cancelamento	Cardápio, total do pedido, status, notificações
Restaurante parceiro	Restaurante, produtos, confirmação/cancelamento	Aviso de novo pedido
Serviço de e-mail/push (SNS/SES)	—	Mensagens de notificação
Provedor de identidade (Cognito, planejado)	Tokens JWT validados no gateway	—

3.2 Contexto Técnico

Canal	Protocolo / formato	Ponto de entrada
Navegador (microfrontends)	HTTPS, JSON	CloudFront (estáticos) e API Gateway (`/bff/web`, `/api`)
App mobile	HTTPS, JSON enxuto	API Gateway (`/bff/mobile`)
Integração entre serviços	Eventos JSON no envelope `{id, tipo, ocorridoEm, dados}`	Amazon EventBridge (local: barramento em memória)
BFF → serviços	HTTP interno com `x-request-id` propagado	API Gateway (`/api/*`)

Canal	Protocolo / formato	Ponto de entrada
Notificações	AWS SDK (SNS Publish)	Lambda `notificacoes`

4. Estratégia de Solução

Estilo arquitetural	Como é aplicado no PedeJá	Onde está no código
Microsserviços + Database per Service	Quatro serviços por capacidade de negócio (Catálogo, Clientes, Pedidos, Notificações), cada um com repositórios próprios. Pedidos mantém *réplicas locais* de restaurantes, produtos e clientes e nunca lê o banco de outro serviço.	`src/services/*`; `infrastructure/` instanciada só na composition root do serviço
Arquitetura Orientada a Eventos	Serviços publicam eventos de integração tipados. Pedidos consome eventos de Catálogo e Clientes para replicar dados (event-carried state transfer). Notificações reage a eventos de pedido.	`shared/contracts/eventos.ts`, `BarramentoEventos`
API Gateway	Entrada única com correlation id, rate limiting, roteamento por prefixo e tradução de erros. Na nuvem: Amazon API Gateway.	`src/gateway/api-gateway.ts`
Backend for Frontend	BFF Web agrega Catálogo + Pedidos para as telas; BFF Mobile devolve payloads enxutos. Os BFFs falam com os serviços só por HTTP.	`src/bff/web`, `src/bff/mobile`
Serverless	Cada serviço e BFF é uma função AWS Lambda. Notificações é 100% orientado a eventos, com handler Lambda real acionado pelo EventBridge.	`services/notificacoes/lambda.ts`
Microfrontends	Shell compõe `<pedeja-catalogo>` e `<pedeja-pedidos>` (Web Components), publicados de forma independente e integrados por eventos DOM.	`frontend/`
Clean Architecture + Vertical Slice	Dentro de cada serviço: `domain` ← `application` (portas) ← `features/<fatia>` (caso de uso + adaptador HTTP) ← `infrastructure`. Cada fatia é uma funcionalidade completa e isolada.	`services/<svc>/{domain,application,features,infrastructure}`
Testes unitários de arquitetura	O grafo de imports é extraído com o compilador TypeScript e as regras rodam no Jest junto dos demais testes. Uma violação quebra o build.	`backend/tests/arquitetura/*.test.ts`

4.1 Software Architecture Canvas

PROPOSTA DE VALOR Pedir comida de restaurantes locais de forma rápida e confiável, com total transparente (subtotal + R$ 7,90) e aviso imediato ao restaurante.	**STAKEHOLDERS PRINCIPAIS** • Clientes • Restaurantes parceiros • Times Catálogo, Clientes, Pedidos e Front • Operação / SRE	**REQUISITOS DE QUALIDADE** • Manutenibilidade (fatias isoladas) • Escalabilidade elástica nos picos • Resiliência a falhas de notificação • Deploy independente • Conformidade verificada por testes
CONTEXTO DE NEGÓCIO Marketplace de delivery com picos nas refeições; restaurantes independentes; notificações por e-mail/push; autenticação via provedor externo.	**FUNCIONALIDADES CENTRAIS** • Catálogo: restaurantes e cardápio • Cadastro de clientes • Pedido com regras e máquina de estados • Notificações por eventos • Telas agregadas (BFF)	**TECNOLOGIAS** • TypeScript, Node.js 24, Express • AWS Lambda, API Gateway, EventBridge • DynamoDB, S3 + CloudFront, SNS/SES • Jest + compilador TS (testes de arquitetura) • PlantUML / C4-PlantUML
COMPONENTES • Shell + MFE Catálogo + MFE Pedidos • API Gateway • BFF Web, BFF Mobile • Serviços Catálogo, Clientes, Pedidos, Notificações • Barramento de eventos; 4 bancos	**DECISÕES CENTRAIS** • Database per Service com réplicas por eventos (ADR-002) • Clean Architecture + Vertical Slice (ADR-004) • Regras verificadas por testes (ADR-005) • Serverless na AWS (ADR-006)	**RISCOS E INFORMAÇÕES EM ABERTO** • Consistência eventual das réplicas • Sem outbox: evento pode se perder se a publicação falhar • Autenticação ainda não implementada • Persistência local apenas em memória

5. Visão de Blocos de Construção

5.1 Visão Sistêmica Geral de Caixa Branca (C4 Nível 2 — Containers)

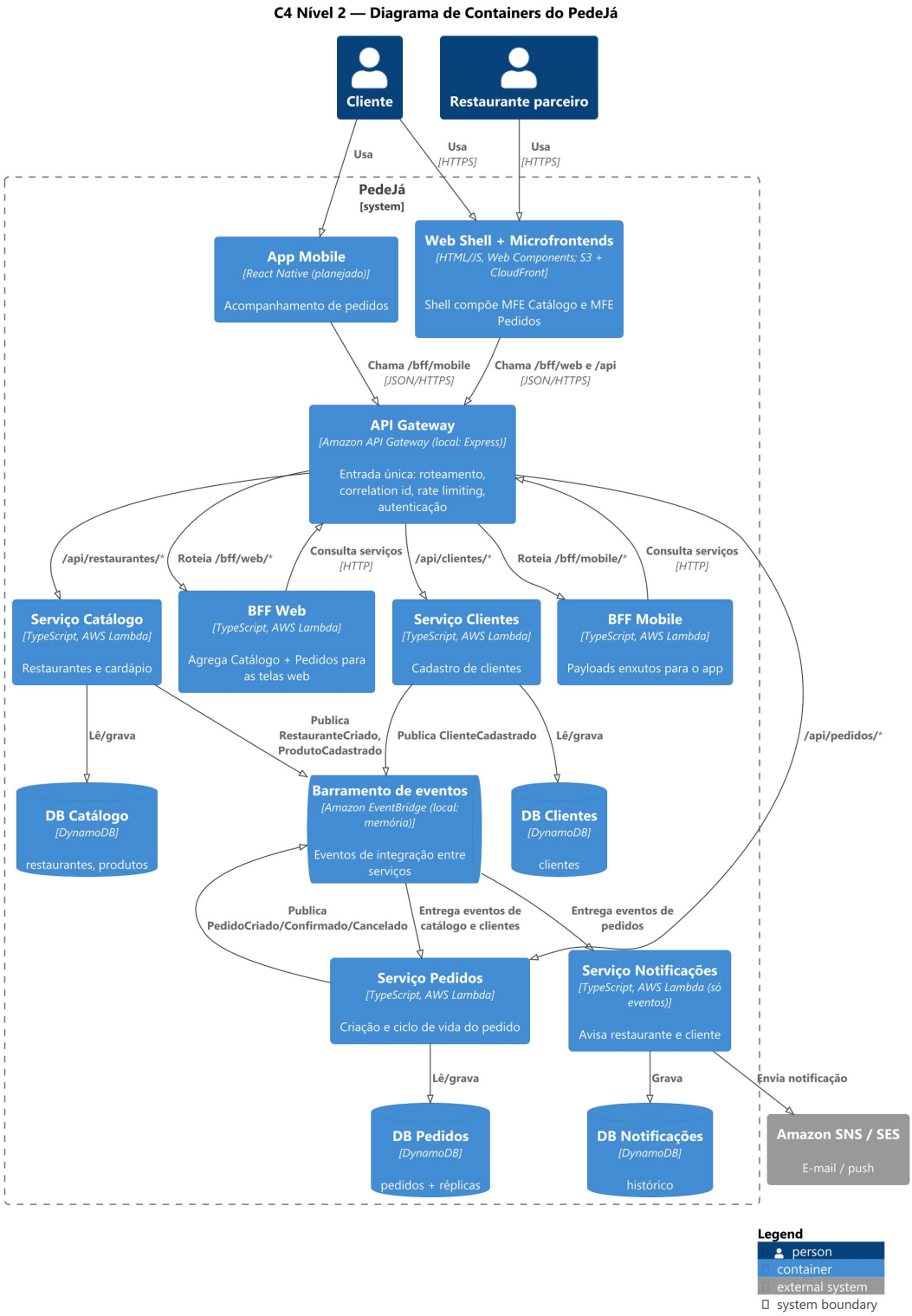


Figura 2 — C4 Nível 2: containers do PedeJá

Motivação

O sistema é decomposto por capacidade de negócio em microsserviços independentes (cada um com seu banco), expostos por um API Gateway e consumidos por BFFs específicos de cada canal. A integração entre serviços é assíncrona, por eventos, para que falhas e picos de um serviço não se propaguem aos demais.

Blocos de Construção Contidos

Os containers e suas responsabilidades estão na tabela abaixo.

Container	Responsabilidade	Dados próprios
Web Shell + Microfrontends	Composição das telas web; cada MFE pertence a um time.	—
API Gateway	Entrada única, correlation id, rate limiting, roteamento, erros.	—
BFF Web / BFF Mobile	Agregação e formatação por canal.	— (sem estado)
Serviço Catálogo	Restaurantes e produtos; publica RestauranteCriado e ProdutoCadastrado.	DB Catálogo
Serviço Clientes	Cadastro e consulta de clientes; publica ClienteCadastrado.	DB Clientes
Serviço Pedidos	Pedido, regras, totais e status; publica eventos de pedido; mantém réplicas.	DB Pedidos (+ réplicas)
Serviço Notificações	Reage a eventos de pedido e notifica restaurante/cliente (sem API HTTP).	DB Notificações
Barramento de eventos	Roteia eventos de integração entre produtores e consumidores.	—

Interfaces Importantes

API REST pública `/api/*` e APIs de canal `/bff/web/*` e `/bff/mobile/*`, ambas pelo API Gateway (contrato de erros `{"erro": "..."}`); eventos de integração definidos em `shared/contracts/eventos.ts` (seção 8.2).

5.2 Nível 2 (C4 Nível 3 — Componentes)

5.2.1 Caixa Branca Serviço Pedidos

Cada caixa "Fatia" é uma pasta em `features/` com seu caso de uso e seu adaptador de entrada. As fatias dependem só das portas (`application`) e do domínio; os repositórios (`infrastructure`) implementam as portas e são instanciados pela composition root.

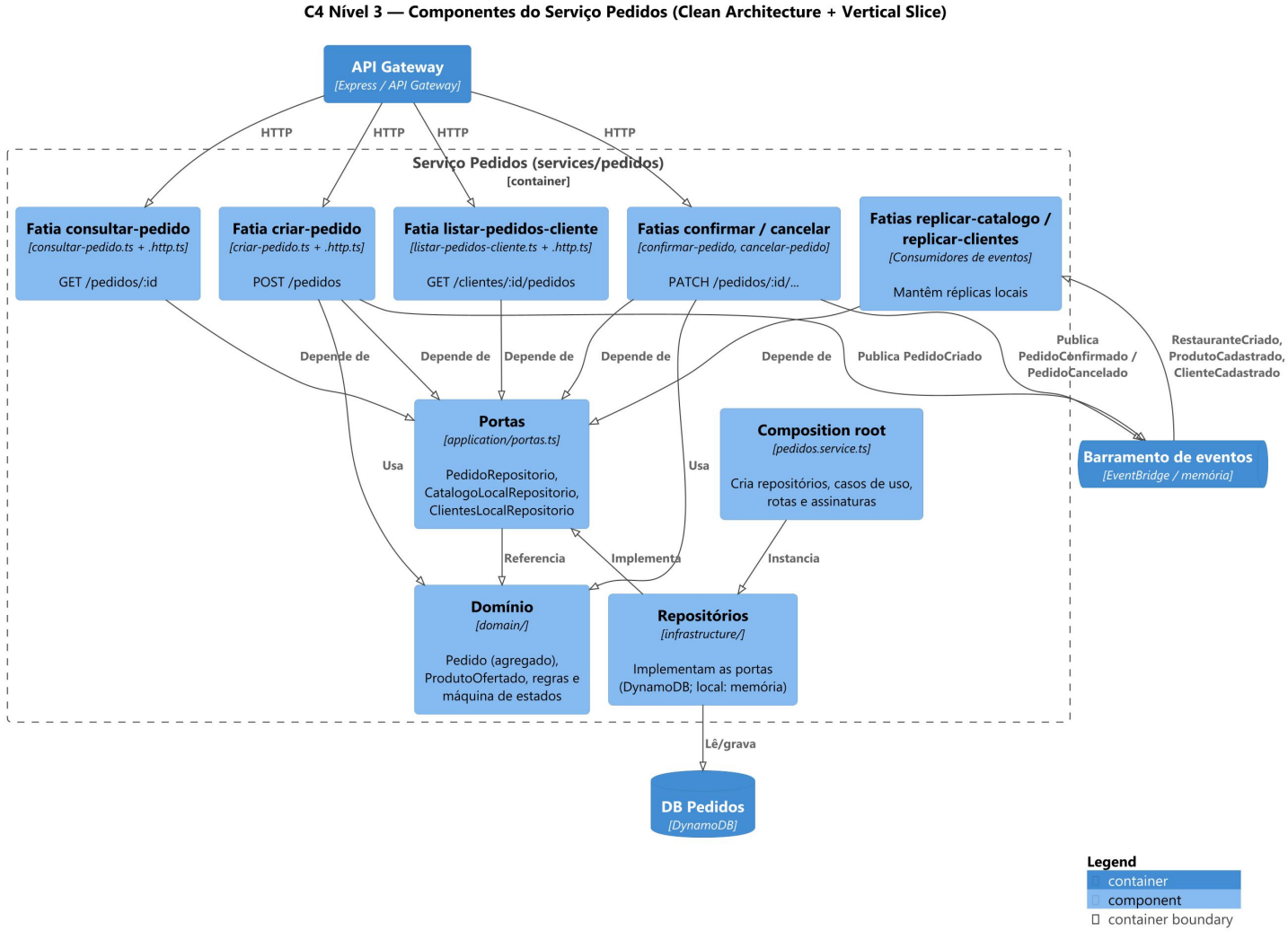


Figura 3 — C4 Nível 3: componentes do Serviço Pedidos

5.2.2 Caixa Branca API Gateway e BFFs

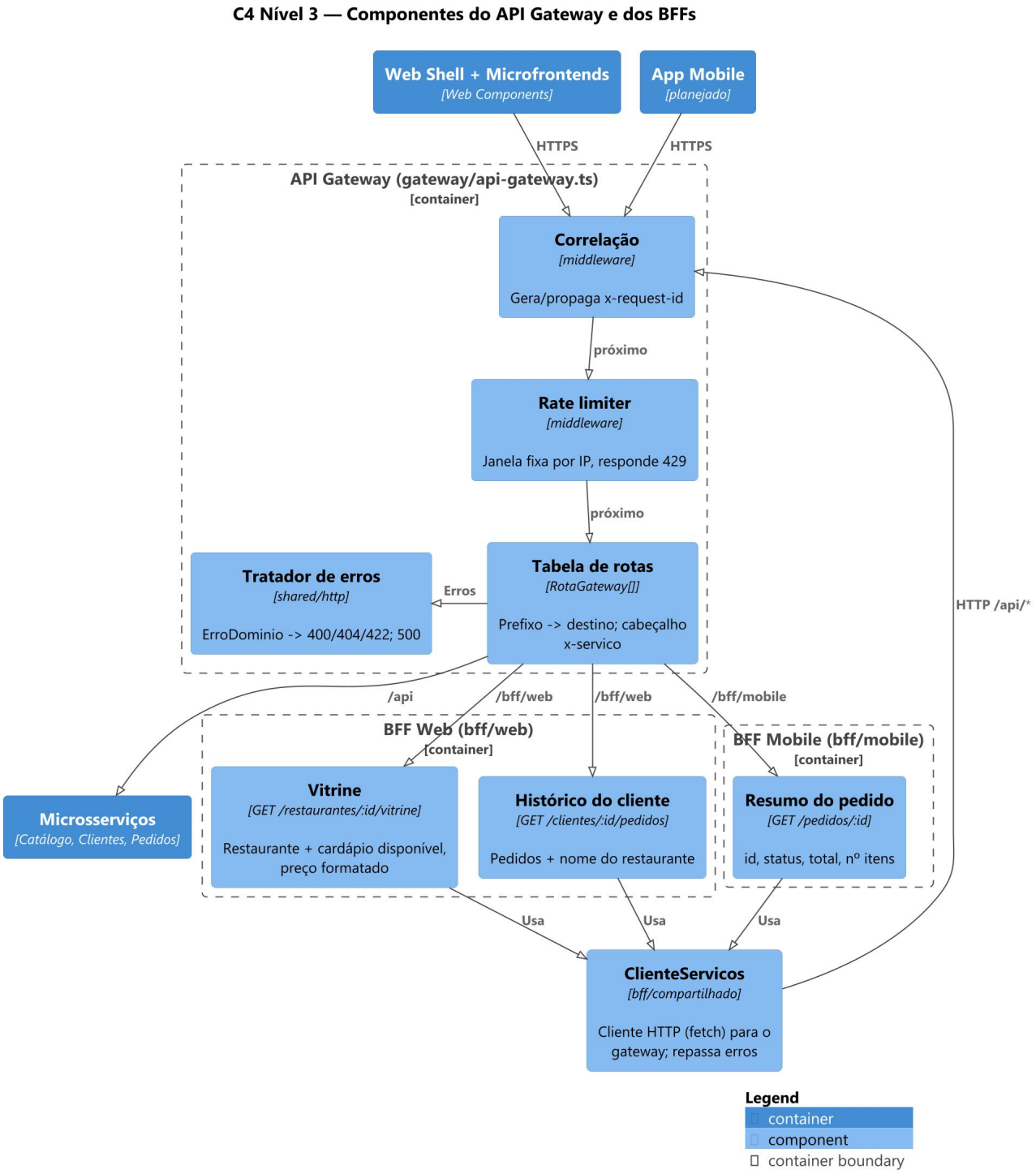


Figura 4 — C4 Nível 3: componentes do API Gateway e dos BFFs

5.2.3 Caixa Branca Serviço Notificações (Serverless)

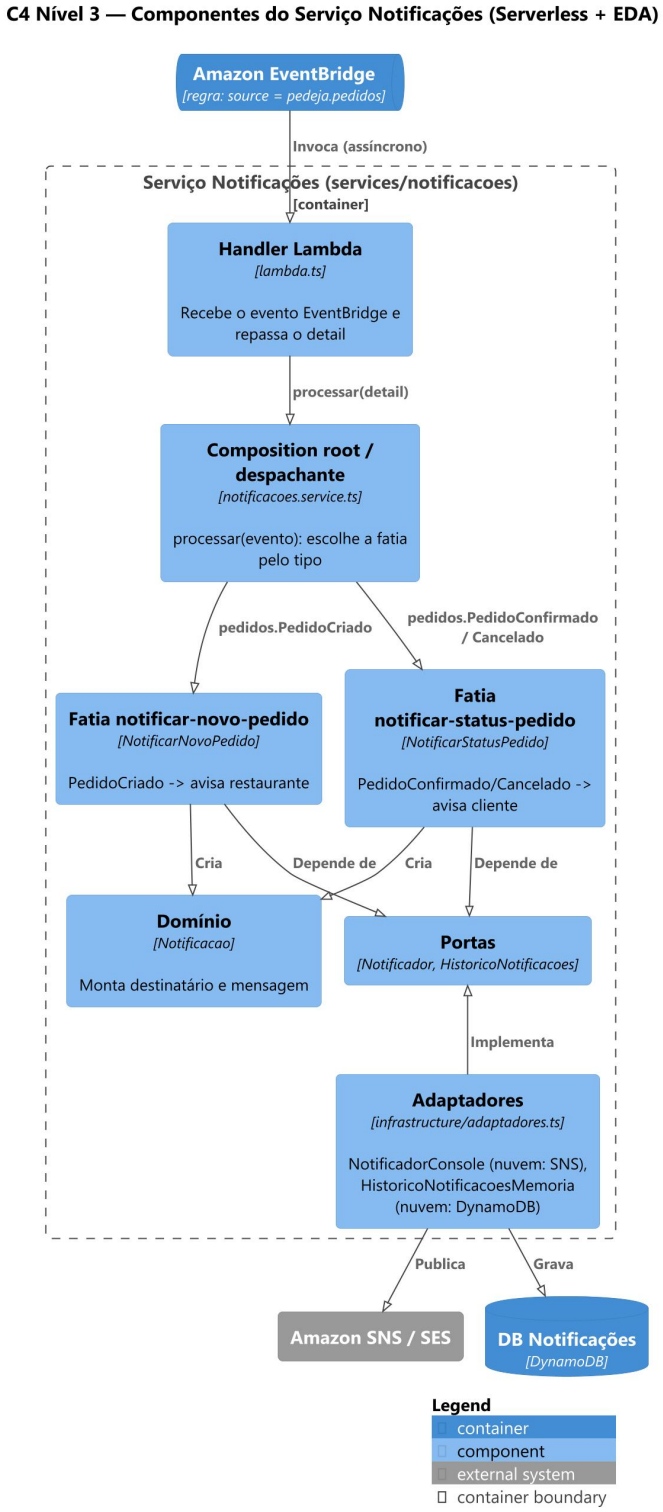


Figura 5 — C4 Nível 3: componentes do Serviço Notificações

5.2.4 Caixa Branca Web Shell (Microfrontends)

O shell (`frontend/index.html`) carrega cada MFE como um módulo independente, que registra um Custom Element. Os MFEs não importam código uns dos outros: comunicam-se por

`CustomEvent` no `window` (`pedeja:restaurante-selecionado`, `pedeja:pedido-criado`), e cada um chama o BFF Web.

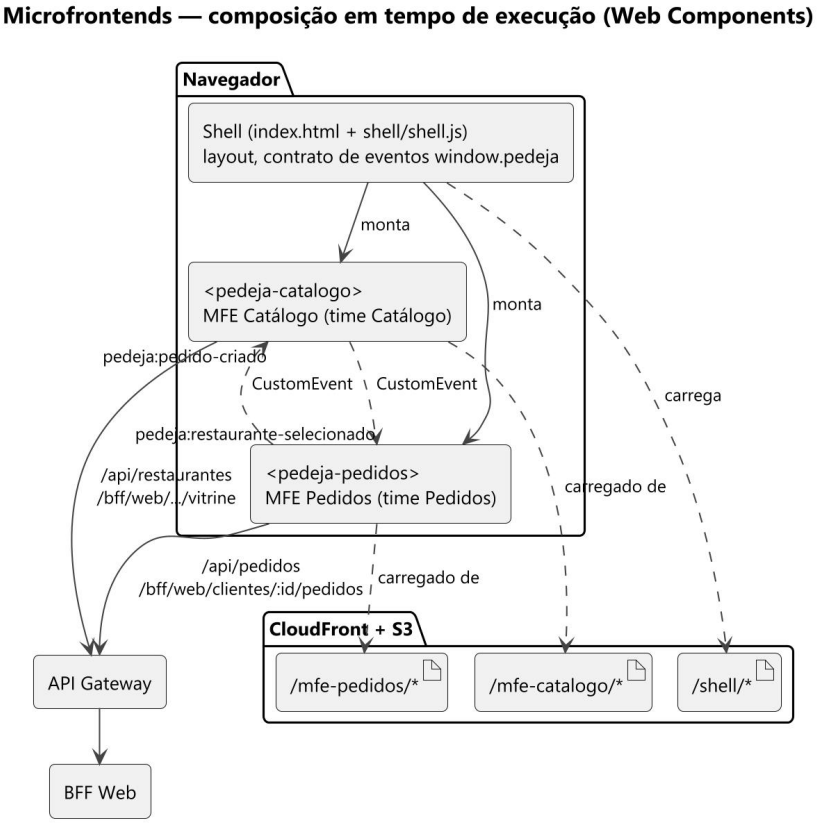


Figura 6 — Composição dos microfrontends

5.2.5 Estrutura do código: Clean Architecture × Vertical Slice

As camadas (linhas) seguem a regra da dependência da Clean Architecture: as setas apontam sempre para dentro. As fatias (colunas) cortam as camadas de entrada e de caso de uso. Domínio, portas e infraestrutura são compartilhados apenas dentro do mesmo serviço.

Camada \ Fatia (Pedidos)	criar-pedido	consultar-pedido	confirmar-pedido	cancelar-pedido	listar-pedidos-cliente	replicar-catalogo / -clientes
Adaptador de entrada	criar-pedido.http.ts	consultar-pedido.http.ts	confirmar-pedido.http.ts	cancelar-pedido.http.ts	listar-pedidos-cliente.http.ts	assinatura no barramento
Caso de uso	CriarPedido	ConsultarPedido	ConfirmarPedido	CancelarPedido	ListarPedidosCliente	ReplicarCatalogo / ReplicarClientes
Portas (application)	PedidoRepositorio · CatalogoLocalRepositorio · ClientesLocalRepositorio · BarramentoEventos · Relogio · GeradorId					
Domínio	Pedido (agregado) · ItemPedido · StatusPedido · ProdutoOfertado · erros de domínio					
Infraestrutura	PedidoRepositorioMemoria · CatalogoLocalRepositorioMemoria · ClientesLocalRepositorioMemoria (nuvem: DynamoDB)					

Árvore de `backend/src`, gerada automaticamente a partir do repositório no build deste documento:

```
backend/src/
├── bff/
│   ├── compartilhado/
│   │   ├── cliente-servicos.ts
│   │   └── http.ts
│   ├── mobile/
│   │   └── bff-mobile.ts
│   └── web/
│       └── bff-web.ts
├── gateway/
│   └── api-gateway.ts
├── services/
│   ├── catalogo/
│   │   ├── application/
│   │   │   └── portas.ts
│   │   ├── domain/
│   │   │   ├── produto.ts
│   │   │   └── restaurante.ts
│   │   ├── features/
│   │   │   ├── cadastrar-produto/
│   │   │   │   ├── cadastrar-produto.http.ts
│   │   │   │   └── cadastrar-produto.ts
│   │   │   ├── criar-restaurante/
│   │   │   │   ├── criar-restaurante.http.ts
│   │   │   │   └── criar-restaurante.ts
│   │   │   ├── listar-cardapio/
│   │   │   │   ├── listar-cardapio.http.ts
│   │   │   │   └── listar-cardapio.ts
│   │   │   ├── listar-restaurantes/
│   │   │   │   ├── listar-restaurantes.http.ts
│   │   │   │   └── listar-restaurantes.ts
│   │   │   └── obter-restaurante/
│   │   │       ├── obter-restaurante.http.ts
│   │   │       └── obter-restaurante.ts
│   │   ├── infrastructure/
│   │   │   └── repositorios-memoria.ts
│   │   └── catalogo.service.ts
│   ├── clientes/
│   │   ├── application/
│   │   │   └── portas.ts
│   │   ├── domain/
│   │   │   └── cliente.ts
│   │   ├── features/
│   │   │   ├── cadastrar-cliente/
│   │   │   │   ├── cadastrar-cliente.http.ts
│   │   │   │   └── cadastrar-cliente.ts
│   │   │   └── obter-cliente/
│   │   │       ├── obter-cliente.http.ts
│   │   │       └── obter-cliente.ts
│   │   ├── infrastructure/
│   │   │   └── repositorio-memoria.ts
│   │   └── clientes.service.ts
│   ├── notificacoes/
│   │   ├── application/
│   │   │   └── portas.ts
│   │   ├── domain/
│   │   │   └── notificacao.ts
│   │   ├── features/
│   │   │   ├── notificar-novo-pedido/
│   │   │   │   └── notificar-novo-pedido.ts
│   │   │   └── notificar-status-pedido/
│   │   │       └── notificar-status-pedido.ts
│   │   ├── infrastructure/
│   │   │   └── adaptadores.ts
│   │   ├── lambda.ts
│   │   └── notificacoes.service.ts
│   └── pedidos/
│       ├── application/
│       │   └── portas.ts
│       ├── domain/
│       │   ├── pedido.ts
│       │   └── produto-ofertado.ts
│       ├── features/
│       │   ├── cancelar-pedido/
│       │   │   ├── cancelar-pedido.http.ts
│       │   │   └── cancelar-pedido.ts
│       │   ├── confirmar-pedido/
│       │   │   ├── confirmar-pedido.http.ts
│       │   │   └── confirmar-pedido.ts
│       │   ├── consultar-pedido/
│       │   │   ├── consultar-pedido.http.ts
│       │   │   └── consultar-pedido.ts
│       │   ├── criar-pedido/
│       │   │   ├── criar-pedido.http.ts
│       │   │   └── criar-pedido.ts
│       │   ├── listar-pedidos-cliente/
│       │   │   ├── listar-pedidos-cliente.http.ts
│       │   │   └── listar-pedidos-cliente.ts
│       │   ├── replicar-catalogo/
│       │   │   └── replicar-catalogo.ts
│       │   └── replicar-clientes/
│       │       └── replicar-clientes.ts
│       ├── infrastructure/
│       │   └── repositorios-memoria.ts
│       └── pedidos.service.ts
├── shared/
│   ├── application/
│   │   └── portas.ts
│   ├── contracts/
│   │   └── eventos.ts
│   ├── domain/
│   │   ├── erros.ts
│   │   └── validacoes.ts
│   ├── http/
│   │   ├── adaptar.ts
│   │   └── tratador-erros.ts
│   └── infrastructure/
│       ├── barramento-eventos-memoria.ts
│       └── plataforma.ts
├── app.ts
└── server.ts
```

5.3 Nível 3 (C4 Nível 4 — Código em UML)

O nível 4 do C4 é documentado com diagramas UML de **classes**, **componentes** e **sequência**. Os diagramas de sequência estão na seção 6 (visão de tempo de execução), conforme o template arc42.

5.3.1 Caixa Branca Pedidos: classes de casos de uso, portas e adaptadores

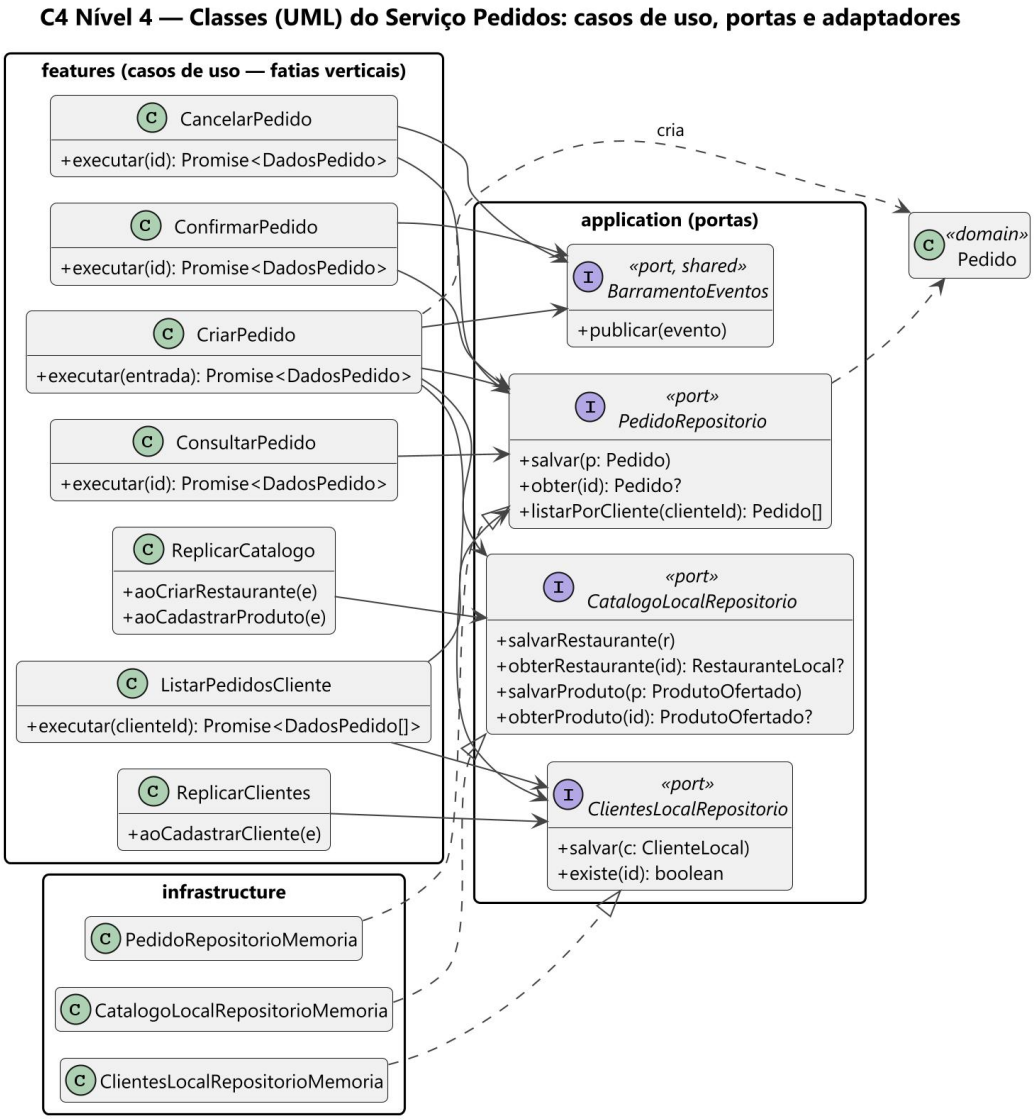


Figura 7 — C4 Nível 4: classes do Serviço Pedidos (casos de uso, portas e adaptadores)

5.3.2 Caixa Branca Pedidos: classes do domínio

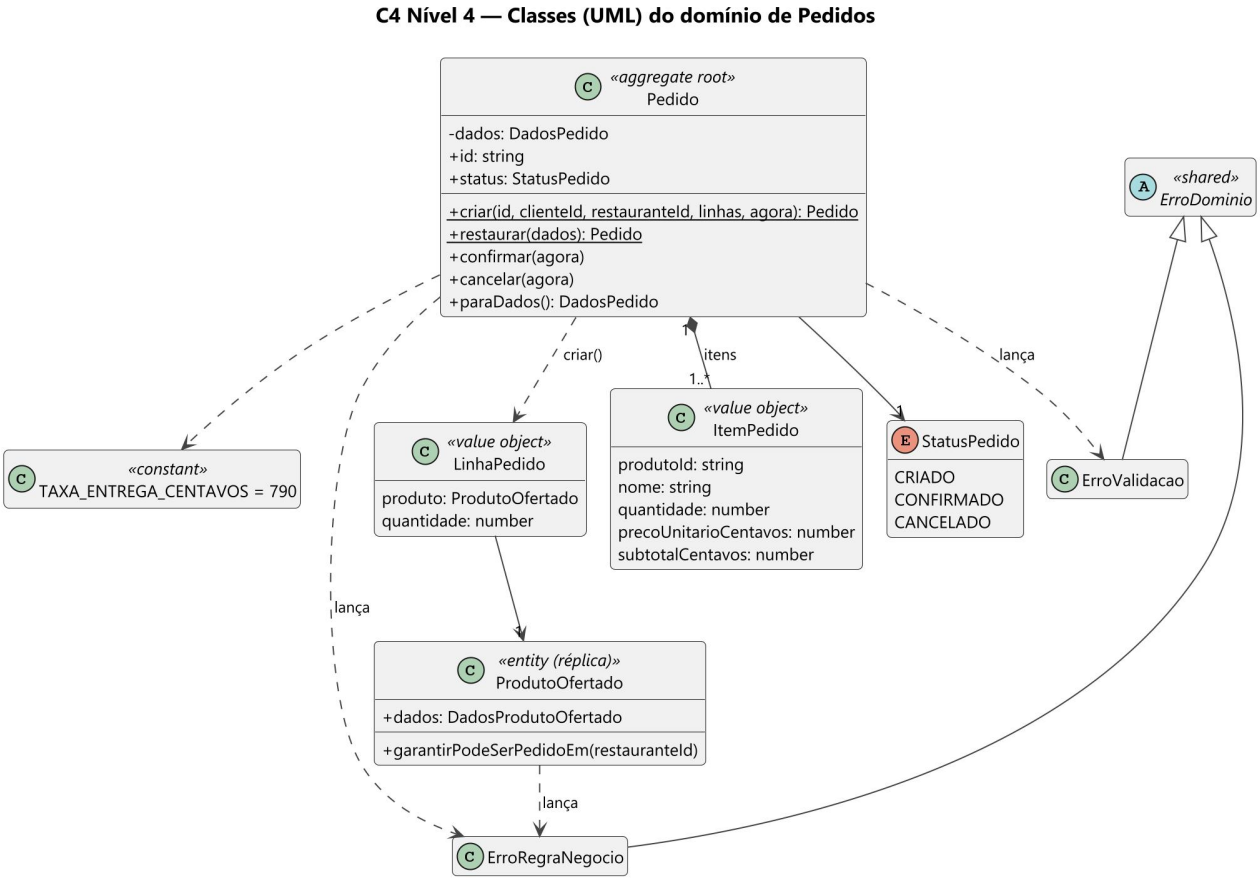


Figura 8 — C4 Nível 4: classes do domínio de Pedidos

5.3.3 Caixa Branca Shared Kernel: classes de erros, portas e eventos

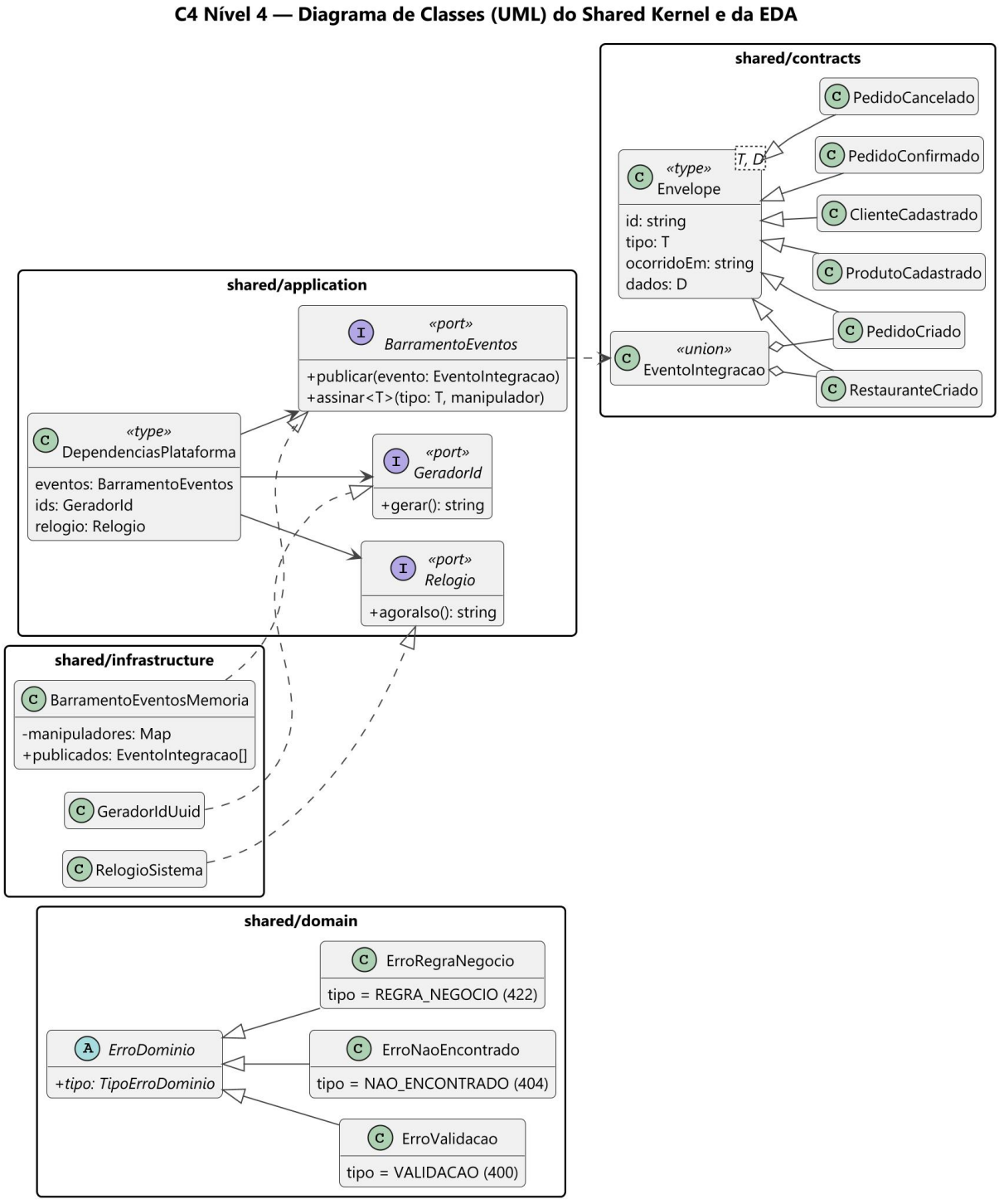


Figura 9 — C4 Nível 4: classes do shared kernel (erros, portas, contratos de eventos)

5.3.4 Caixa Branca Pedidos: componentes UML

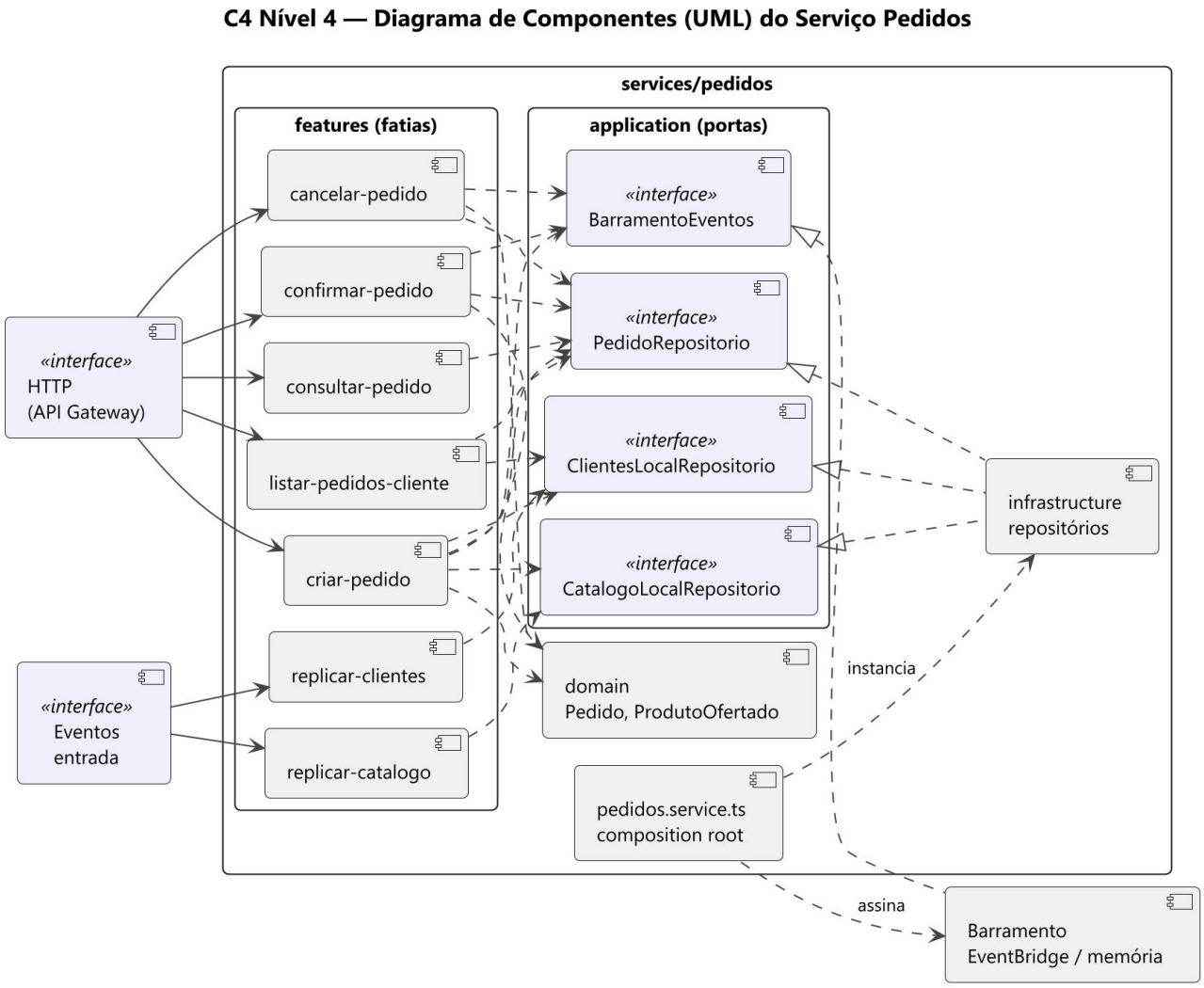


Figura 10 — C4 Nível 4: diagrama de componentes UML do Serviço Pedidos

6. Visão de Tempo de Execução

Diagramas de sequência UML (C4 Nível 4) dos cenários mais importantes. Todos são cobertos por testes automatizados (`tests/e2e` e `tests/unit`).

6.1 Criar pedido e notificar o restaurante

O caso de uso valida o formato (400), confere existência nas réplicas locais (404) e delega ao agregado as regras de negócio (422) e o cálculo do total. Persistido o pedido, publica `pedidos.PedidoCriado`; a notificação acontece de forma assíncrona em outro serviço.

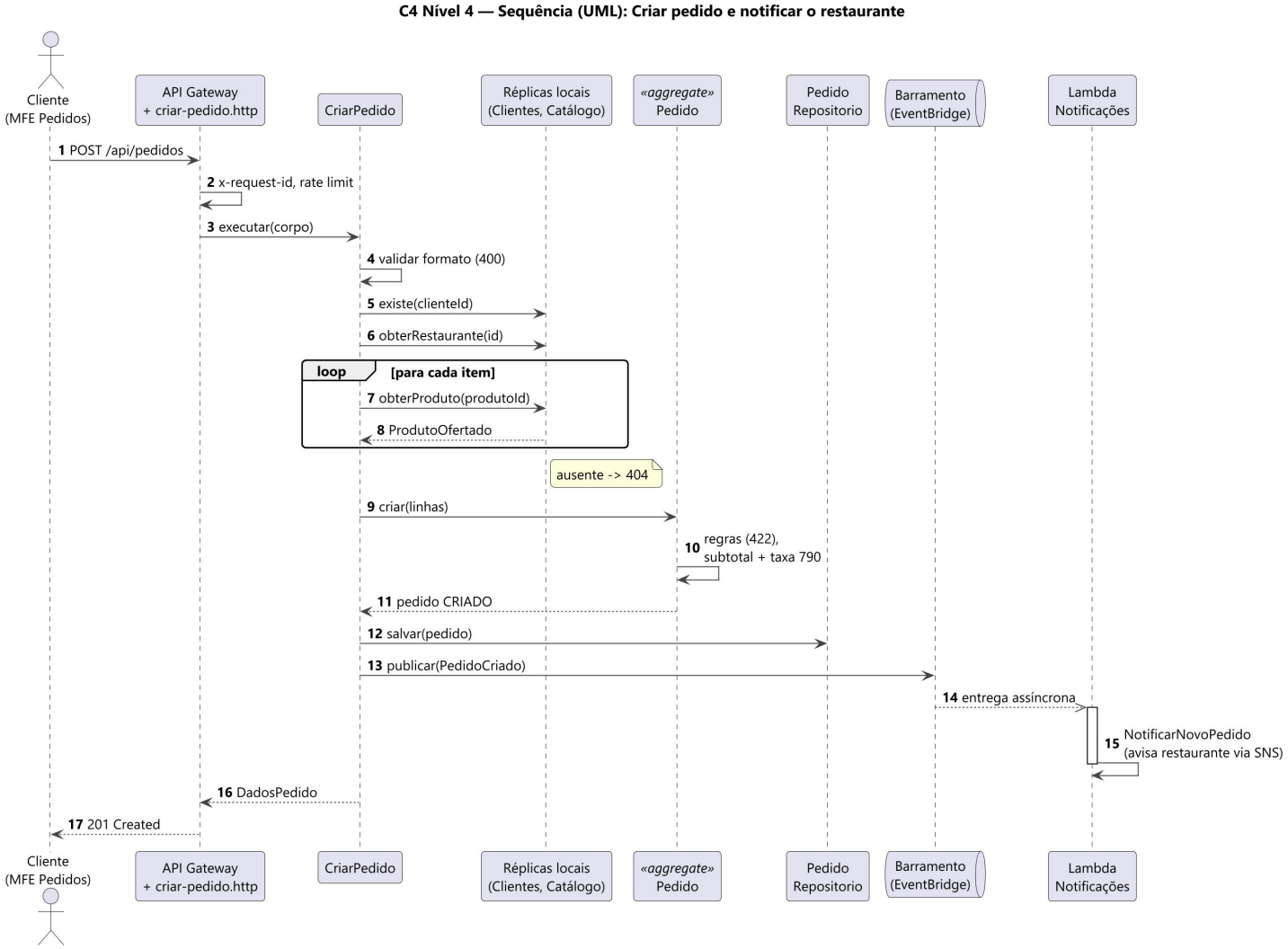


Figura 11 — Sequência: criar pedido

6.2 Cadastro de produto e replicação para Pedidos (EDA)

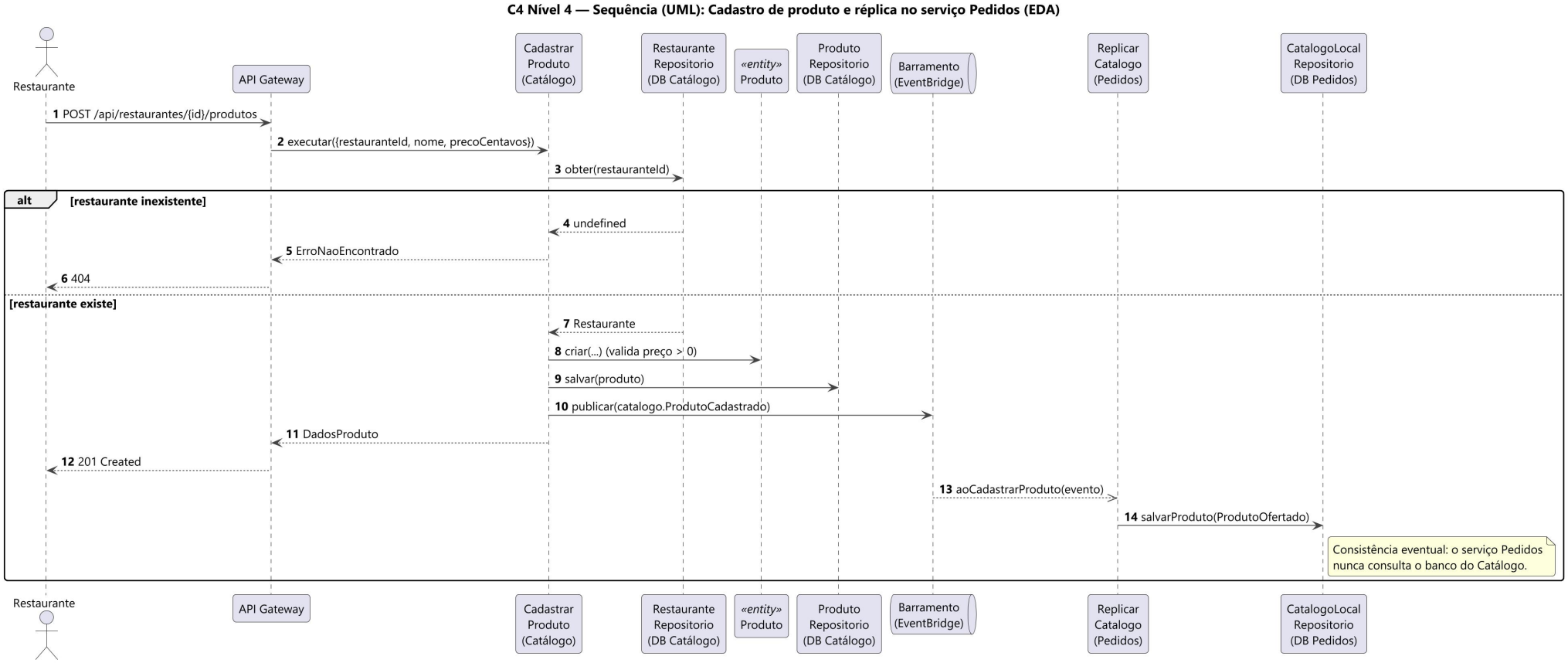


Figura 12 — Sequência: cadastro de produto e réplica por evento

6.3 Vitrine do restaurante via BFF Web

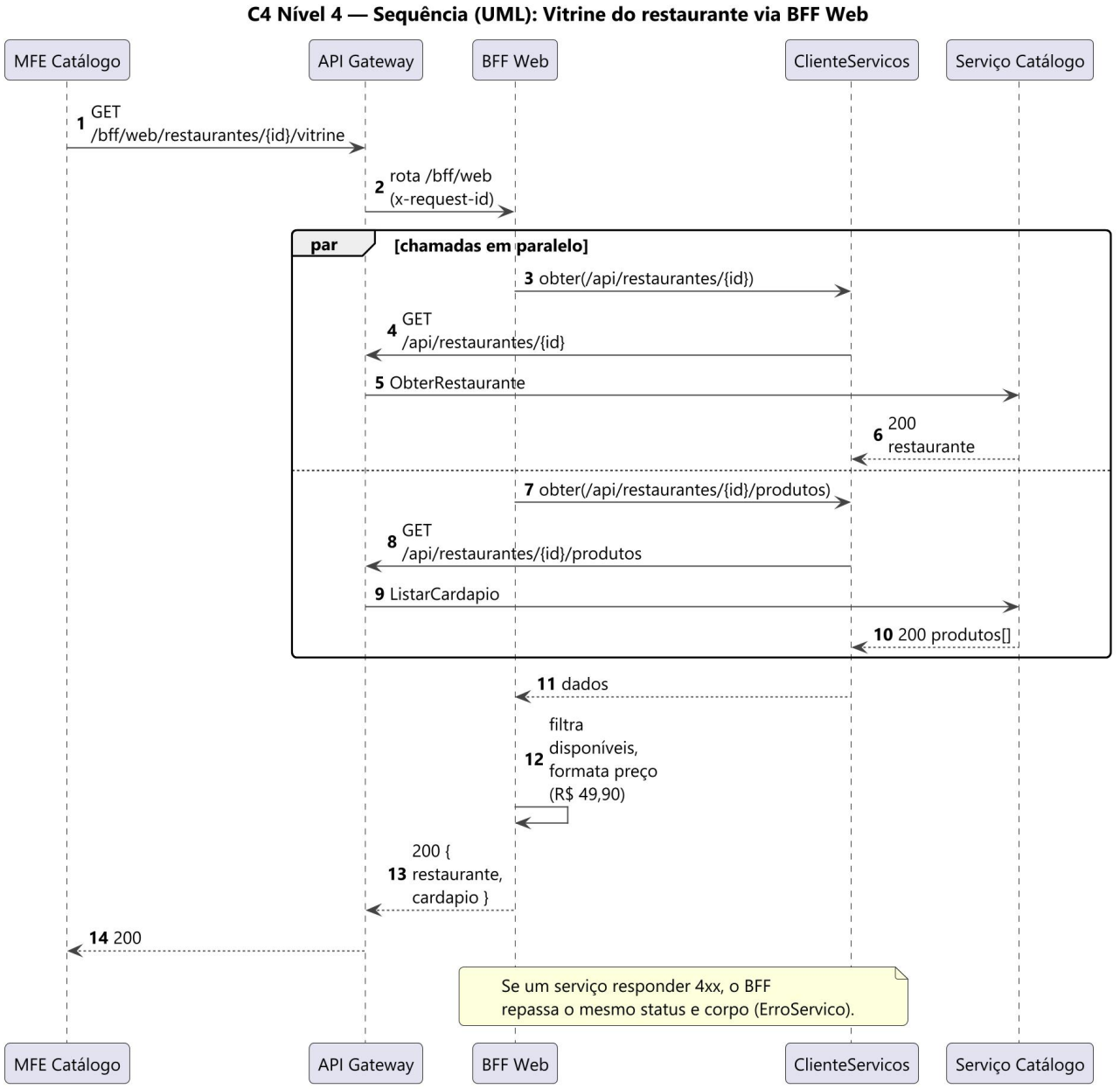


Figura 13 — Sequência: agregação no BFF Web

6.4 Confirmar / cancelar pedido

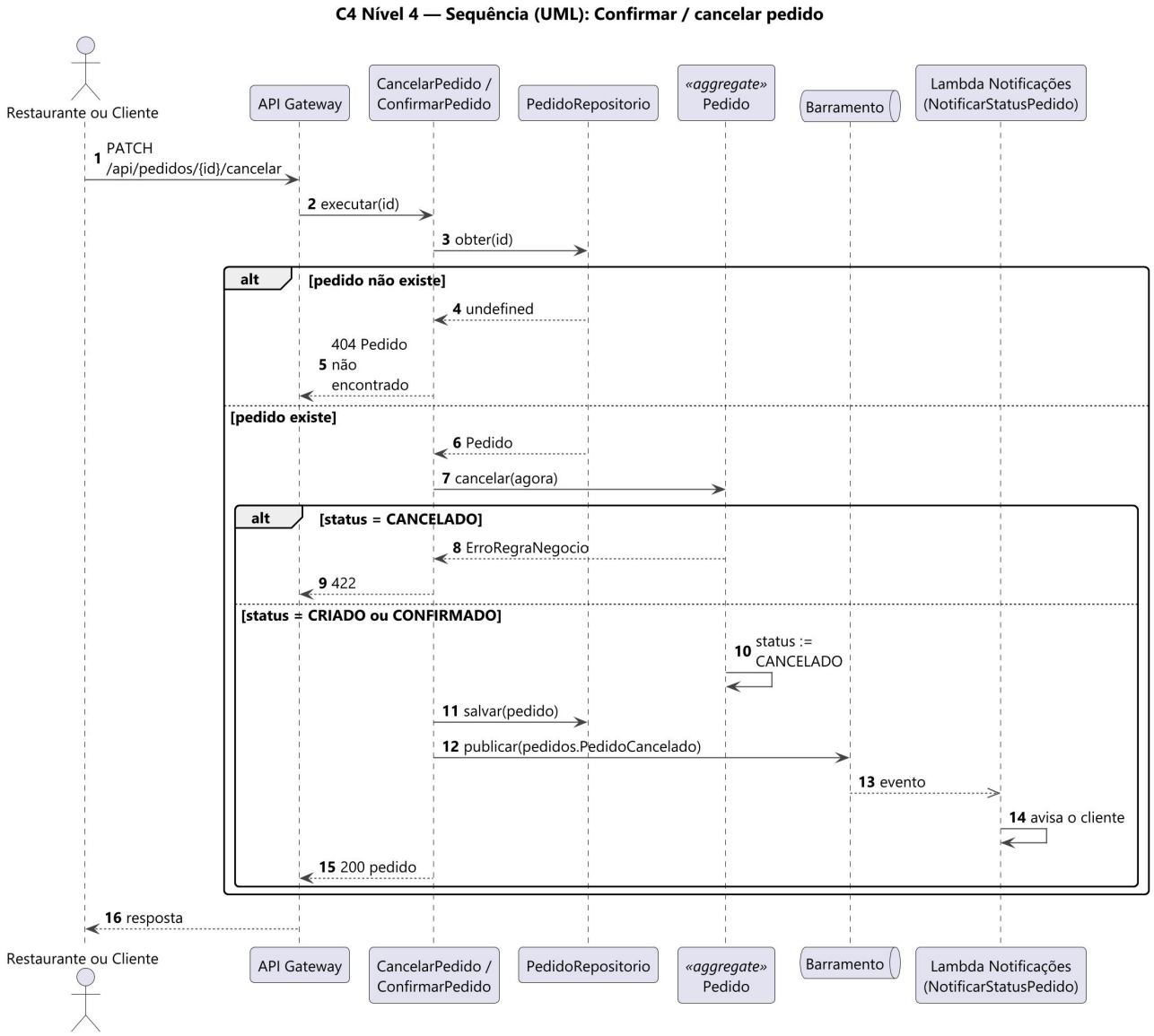


Figura 14 — Sequência: cancelar pedido

6.5 Ciclo de vida do pedido

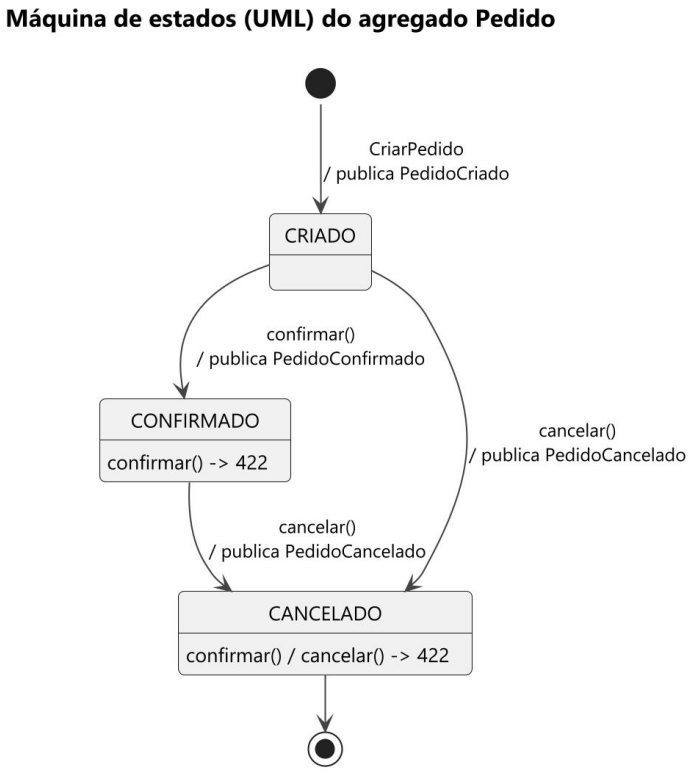


Figura 15 — Máquina de estados do agregado Pedido

7. Visão de Implantação

7.1 Nível de Infraestrutura 1 (AWS — produção)

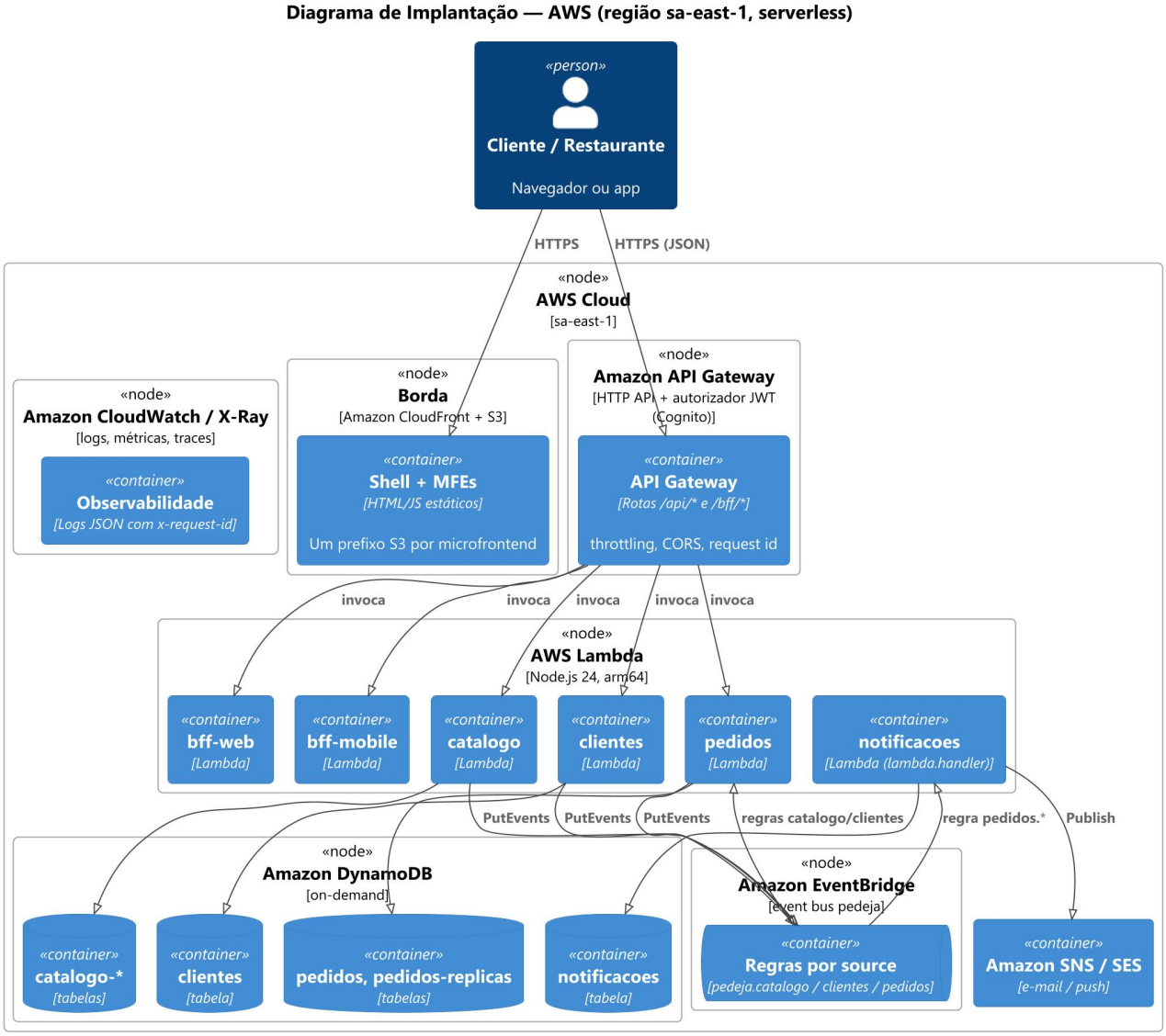


Figura 16 — Implantação na AWS (serverless)

Motivação
Modelo serverless na AWS: sem servidores para administrar, escala automática por função e custo proporcional ao uso, adequado à carga com picos nos horários de refeição.

Características de Qualidade e/ou Desempenho
Cada função escala de forma independente; DynamoDB on-demand acompanha o volume; o EventBridge faz retentativas e envia para DLQ em caso de falha do consumidor; inicialização fora do handler reduz o custo de cold start.

Mapeamento de Blocos de Construção para Infraestrutura
Conforme a tabela abaixo.

Bloco de construção	Artefato	Infraestrutura

Bloco de construção	Artefato	Infraestrutura
Shell + MFEs	`frontend/**`	S3 (um prefixo por MFE) + CloudFront
API Gateway	Tabela de rotas (prefixo → função)	Amazon API Gateway HTTP API + autorizador JWT
BFF Web, BFF Mobile, Catálogo, Clientes, Pedidos	Composition root de cada um, empacotada como Lambda	AWS Lambda (Node.js 24)
Notificações	`services/notificacoes/lambda.ts` (`handler`)	AWS Lambda acionada por regra do EventBridge
Barramento	Adaptador do `BarramentoEventos` (PutEvents)	Amazon EventBridge (bus `pedeja`)
Bancos por serviço	Adaptadores dos repositórios	Amazon DynamoDB (tabelas separadas, IAM por função)

7.2 Nível de Infraestrutura 2

7.2.1 Ambiente local de desenvolvimento

Para desenvolvimento e testes, `app.ts` (composition root local) sobe gateway, BFFs e serviços num único processo, ligados pelo barramento em memória. Os casos de uso são os mesmos da nuvem; só mudam os adaptadores.

Diagrama de Implantação (UML) — Ambiente local / desenvolvimento

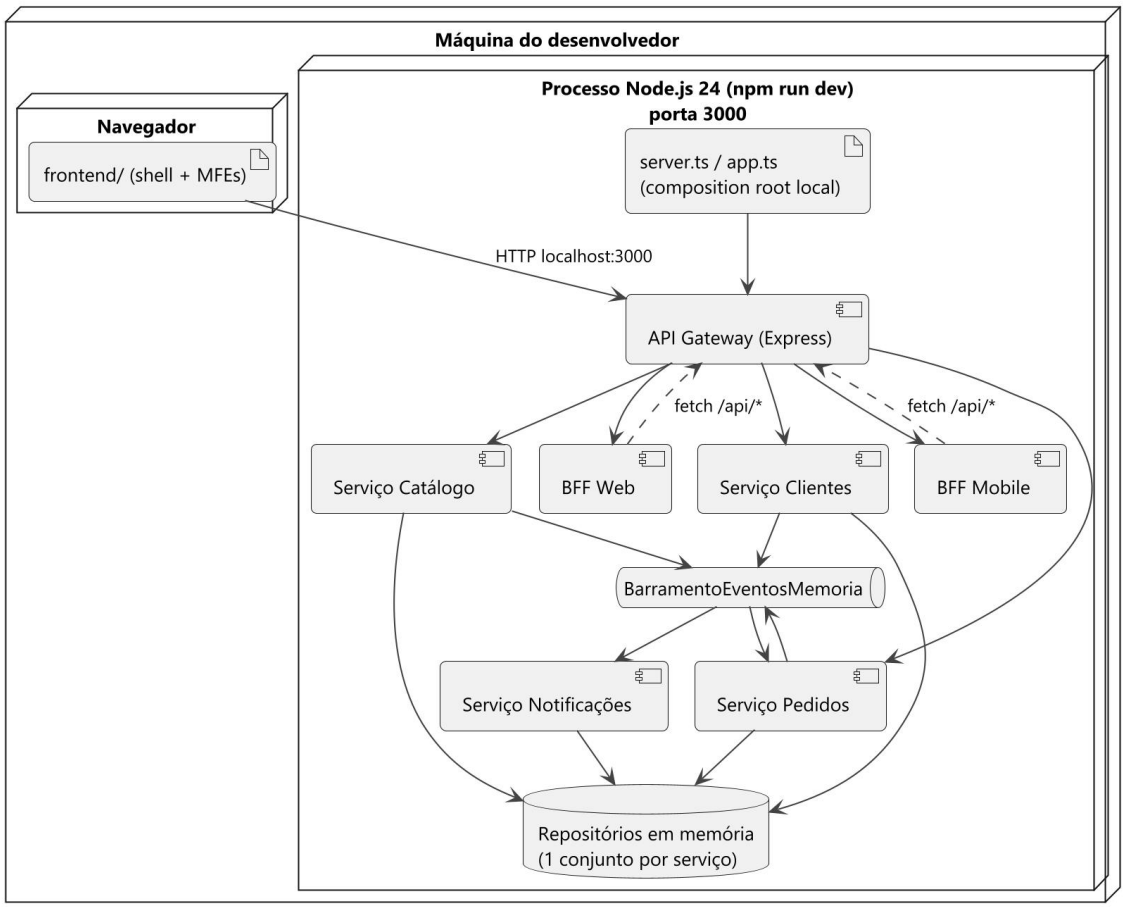


Figura 17 — Implantação local (desenvolvimento)

8. Conceitos Transversais

8.1 Modelo de dados e Database per Service (DER)

Cada serviço é dono do seu banco. Relações entre bancos são apenas *referências lógicas* (ids), sem chave estrangeira física. O Serviço Pedidos guarda réplicas mínimas (restaurante, produto, cliente) alimentadas por eventos e faz snapshot de nome e preço em `item_pedido`, para que mudanças futuras no cardápio não alterem pedidos já feitos.

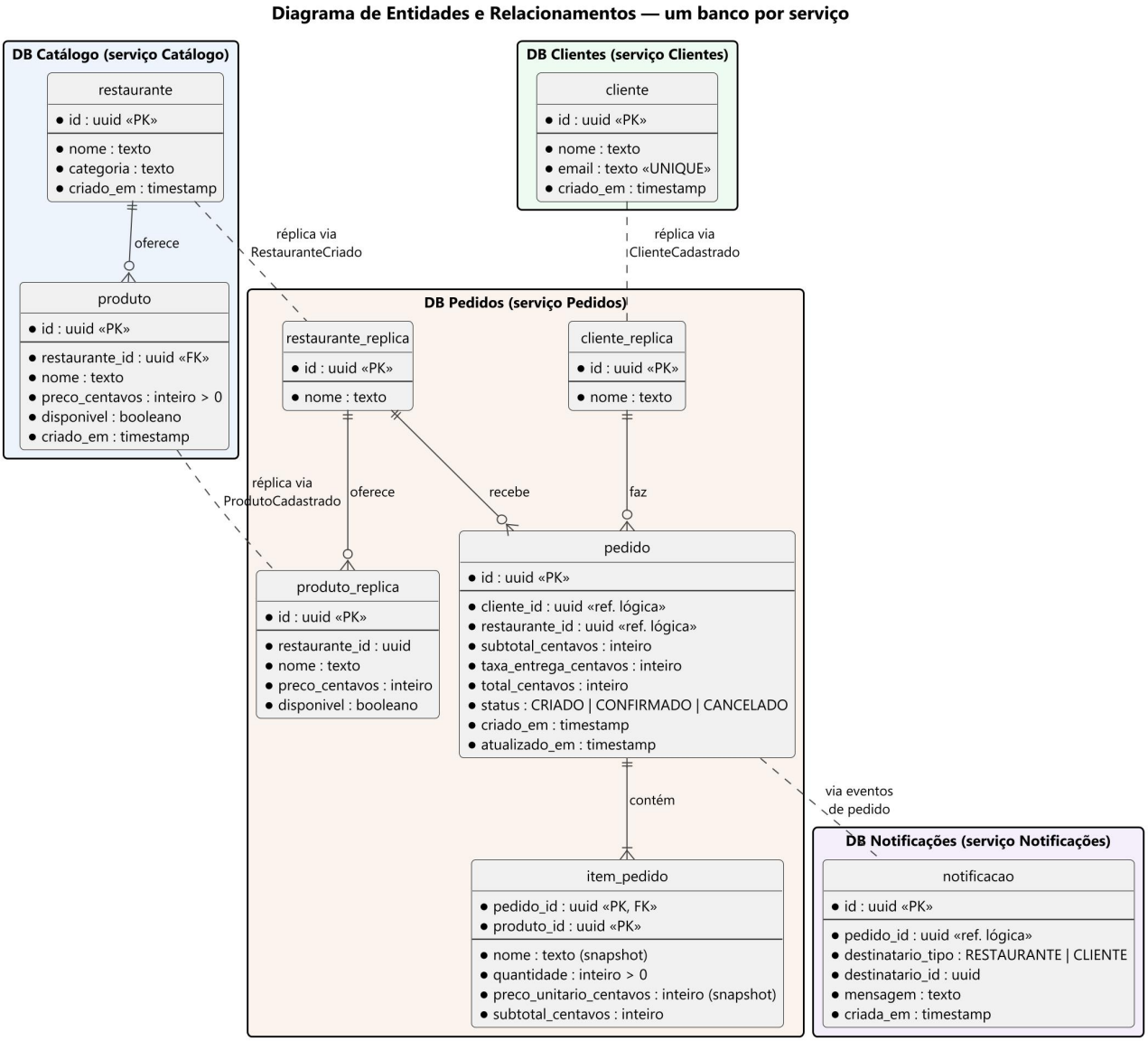


Figura 18 — Diagrama de Entidades e Relacionamentos (um banco por serviço)

8.2 Arquitetura orientada a eventos

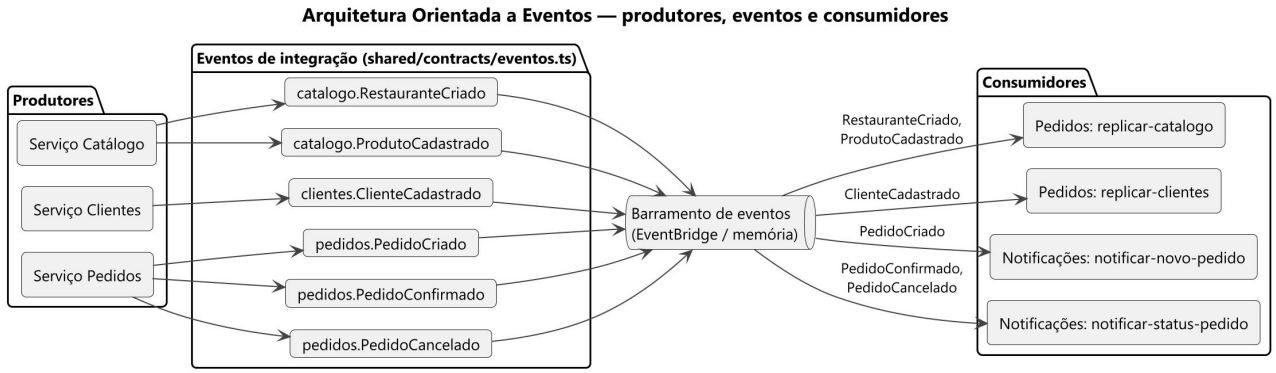


Figura 19 — Produtores, eventos de integração e consumidores

Evento	Produtor	Consumidores	Dados
`catalogo.RestauranteCriado`	Catálogo	Pedidos (réplica)	restauranteId, nome, categoria
`catalogo.ProdutoCadastrado`	Catálogo	Pedidos (réplica)	produtoId, restauranteId, nome, precoCentavos, disponivel
`clientes.ClienteCadastrado`	Clientes	Pedidos (réplica)	clienteId, nome, email
`pedidos.PedidoCriado`	Pedidos	Notificações	pedidoId, clienteId, restauranteId, restauranteNome, quantidadeItens, totalCentavos
`pedidos.PedidoConfirmado`	Pedidos	Notificações	pedidoId, clienteId, restauranteId
`pedidos.PedidoCancelado`	Pedidos	Notificações	pedidoId, clienteId, restauranteId

Os contratos ficam em `shared/contracts/eventos.ts` e são o único acoplamento permitido entre serviços. A falha de um consumidor é registrada e não afeta o produtor; na nuvem, o EventBridge faz retentativas e envia para uma DLQ.

8.3 Regras de Clean Architecture e Vertical Slice

- **domain**: entidades, agregados e regras; não importa nada fora de `domain` nem frameworks.
- **application**: portas (interfaces) de repositórios e plataforma (`BarramentoEventos`, `Relogio`, `GeradorId`).
- **features/<fatia>**: `<fatia>.ts` é o caso de uso (sem HTTP); `<fatia>.http.ts` é o adaptador de entrada, que só traduz HTTP e delega. Uma fatia não importa outra.
- **infrastructure**: implementações das portas, instanciadas somente pela composition root do próprio serviço.
- **composition roots**: `<servico>.service.ts` (local), `lambda.ts` (serverless) e `app.ts` (processo local completo).

8.4 Testes unitários de arquitetura

Os testes de arquitetura leem os imports de todos os arquivos de `src/` com `ts.preProcessFile`, do compilador TypeScript, e verificam as regras abaixo. Cada regra exige ao menos um arquivo correspondente, para não passar "no vazio". Durante o

desenvolvimento, três violações foram inseridas de propósito (domínio importando express, fatia importando outra fatia, serviço importando outro serviço) e os testes falharam como esperado. Lista gerada a partir dos arquivos de teste:

Arquivo	Grupo	Regra verificada
`clean-architecture.test.ts`	Clean Architecture	domain não depende de nenhuma outra camada (só de outros arquivos de domain)
`clean-architecture.test.ts`	Clean Architecture	domain não importa frameworks nem APIs de infraestrutura do Node
`clean-architecture.test.ts`	Clean Architecture	application (portas) depende apenas de domain e de contratos
`clean-architecture.test.ts`	Clean Architecture	casos de uso não dependem de infrastructure
`clean-architecture.test.ts`	Clean Architecture	casos de uso não conhecem HTTP (express ou shared/http)
`clean-architecture.test.ts`	Clean Architecture	adaptadores HTTP só delegam ao caso de uso da própria fatia (não acessam repositórios nem domínio)
`clean-architecture.test.ts`	Clean Architecture	infrastructure não é importada por ninguém além das composition roots
`microsservicos.test.ts`	Microsserviços e Database per Service	um microsserviço não importa código de outro microsserviço
`microsservicos.test.ts`	Microsserviços e Database per Service	a única dependência compartilhada é o shared kernel (erros, portas, contratos de eventos, adaptadores genéricos)
`microsservicos.test.ts`	Microsserviços e Database per Service	o shared kernel não conhece serviços, BFFs nem o gateway
`microsservicos.test.ts`	Microsserviços e Database per Service	o banco (infrastructure) de cada serviço só é instanciado pela composition root do próprio serviço
`microsservicos.test.ts`	Arquitetura Orientada a Eventos	serviços só se comunicam pelos contratos de eventos (shared/contracts)
`microsservicos.test.ts`	Backend for Frontend	BFFs não importam código de microsserviços (falam com eles só via HTTP/gateway)
`microsservicos.test.ts`	Backend for Frontend	um BFF não importa outro BFF (apenas a pasta compartilhado)
`microsservicos.test.ts`	API Gateway	o gateway não conhece serviços nem BFFs (recebe as rotas por injeção)
`microsservicos.test.ts`	Serverless	o handler Lambda de Notificações não depende de Express
`microsservicos.test.ts`	Serverless	o serviço de Notificações não tem API HTTP (é acionado só por eventos)
`vertical-slice.test.ts`	Vertical Slice	uma fatia não importa código de outra fatia
`vertical-slice.test.ts`	Vertical Slice	domain e application não dependem de nenhuma fatia
`vertical-slice.test.ts`	Vertical Slice	toda fatia tem um caso de uso com o mesmo nome da pasta
`vertical-slice.test.ts`	Vertical Slice	arquivos de uma fatia ficam todos na pasta da fatia (sem subpastas por camada técnica)
`vertical-slice.test.ts`	Vertical Slice	fatias com entrada HTTP pertencem a serviços expostos pelo gateway

8.5 Tratamento de erros

O domínio lança `ErroValidacao`, `ErroNaoEncontrado` ou `ErroRegraNegocio`, que não conhecem HTTP. O adaptador `shared/http/tratador-erros.ts` mapeia esses erros para 400, 404 e 422; JSON malformado vira 400 e o restante, 500. Os BFFs repassam ao front o status e o corpo do serviço chamado (`ErroServico`).

8.6 Segurança, observabilidade e configuração

- **Correlation id**: o gateway gera ou propaga `x-request-id`, e os BFFs o repassam nas chamadas aos serviços.
- **Rate limiting**: janela fixa por IP no gateway (HTTP 429); na nuvem, throttling do API Gateway.
- **Roteamento observável**: cabeçalho `x-servico` indica o destino da requisição.
- **Autenticação** (planejada): autorizador JWT do API Gateway com Amazon Cognito.
- **Menor privilégio**: cada Lambda tem papel IAM com acesso apenas às próprias tabelas, o que reforça o Database per Service.
- **Configuração**: `PORT` e `SERVICOS_URL` por variáveis de ambiente.

9. Decisões Arquiteturais

ADR	Decisão	Contexto e justificativa	Consequências
001	Decompor em 4 microsserviços por capacidade de negócio	Times e ritmos de mudança distintos; picos concentrados em Pedidos.	+ escala e deploy independentes; − mais peças para operar.
002	Database per Service com réplicas alimentadas por eventos	Pedidos precisa de dados de Catálogo e Clientes sem acoplamento síncrono.	+ autonomia e resiliência; − consistência eventual.
003	Integração assíncrona por eventos (EventBridge)	Notificação não pode bloquear nem derrubar a criação do pedido.	+ desacoplamento; − rastreamento distribuído mais difícil (mitigado com x-request-id).
004	Clean Architecture dentro de cada serviço, organizada em Vertical Slices	O código inicial misturava camadas nas rotas.	+ casos de uso testáveis sem HTTP/banco; + mudanças localizadas em uma pasta.
005	Verificar a arquitetura com testes automatizados	Regras só documentadas tendem a ser violadas.	+ violações quebram o build; − convenção de pastas precisa ser seguida.
006	Serverless (Lambda) como modelo de execução	Carga com picos e vales; equipe pequena.	+ custo proporcional ao uso, sem servidores; − cold start (mitigado com init fora do handler).
007	API Gateway como entrada única + BFF por canal	Web e mobile precisam de formatos diferentes; preocupações transversais centralizadas.	+ fronts simples; − mais um salto de rede.
008	Microfrontends com Web Components e eventos DOM	Times de Catálogo e Pedidos publicam telas de forma independente.	+ sem framework obrigatório; − consistência visual depende do shell.
009	Execução local em processo único com adaptadores em memória	Desenvolver e testar sem conta AWS.	+ feedback rápido (testes em segundos); − não exercita latência real.

10. Requisitos de Qualidade

10.1 Árvore de Qualidade

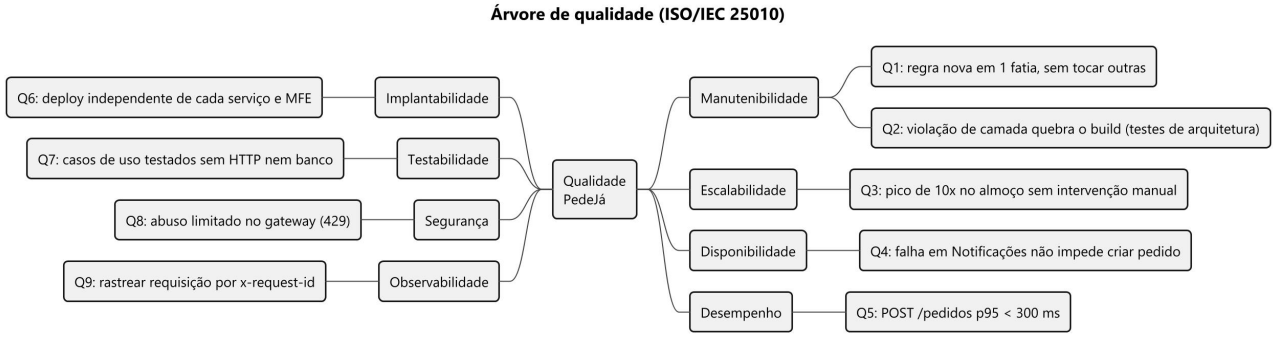


Figura 20 — Árvore de qualidade

10.2 Cenários de Qualidade

ID	Atributo	Estímulo	Resposta esperada	Como é atendido
Q1	Manutenibilidade	Nova regra de cancelamento	Alteração restrita a `cancelar-pedido` e ao agregado	Vertical Slice + domínio rico
Q2	Conformidade	Dev importa express no domínio	Build falha no CI	Testes de arquitetura
Q3	Escalabilidade	10x pedidos às 12h	Sem intervenção manual	Lambda + DynamoDB on-demand
Q4	Disponibilidade	Serviço Notificações fora do ar	Pedidos continuam sendo criados (201)	EDA assíncrona; falha do consumidor isolada
Q5	Desempenho	POST /pedidos em carga normal	p95 < 300 ms	Leituras só em réplicas locais (sem chamadas síncronas a outros serviços)
Q6	Implantabilidade	Novo layout do MFE Catálogo	Deploy sem republicar os demais	Microfrontends + serviços independentes
Q7	Testabilidade	Testar regra de total	Teste unitário sem HTTP nem banco	Portas + fakes (`tests/unit`)
Q8	Segurança	Cliente dispara milhares de requisições	HTTP 429 após o limite	Rate limiting no gateway
Q9	Observabilidade	Investigar um pedido com erro	Requisição rastreada ponta a ponta	`x-request-id` propagado

11. Riscos e Débitos Técnicos

#	Risco / débito	Impacto	Mitigação planejada
R1	Consistência eventual: pedido criado logo após cadastrar produto pode não achar a réplica.	Médio	Retentativa no front; monitorar o atraso de entrega dos eventos.
R2	Sem transactional outbox: falha entre salvar e publicar perde o evento.	Alto	Outbox com DynamoDB Streams → EventBridge.
R3	Autenticação/autorização ainda não implementadas.	Alto	Autorizador JWT (Cognito) no API Gateway.
R4	Persistência local em memória: dados se perdem ao reiniciar.	Baixo (dev)	Adaptadores DynamoDB atrás das mesmas portas.
R5	Disponibilidade e preço do produto não se atualizam (não há evento de alteração).	Médio	Fatia `atualizar-produto` e evento `ProdutoAlterado`.
D1	Rate limiter em memória não funciona com várias instâncias.	Baixo	Usar o throttling nativo do API Gateway em produção.
D2	App mobile apenas planejado (BFF pronto).	Baixo	Roadmap.

12. Glossário

Termo	Definição
Agregado	Conjunto de objetos de domínio tratado como unidade de consistência (ex.: Pedido e seus itens).
BFF	Backend for Frontend: backend dedicado a um canal (web, mobile) que agrega e molda dados.
C4 Model	Modelo de diagramas em 4 níveis: Contexto, Containers, Componentes e Código.
Composition root	Único ponto onde as implementações concretas são instanciadas e injetadas.
Database per Service	Cada microsserviço é dono exclusivo de seus dados.
EDA	Event-Driven Architecture: integração por eventos publicados em um barramento.
Evento de integração	Fato de negócio publicado por um serviço para outros (ex.: `pedidos.PedidoCriado`).
Fatia (Vertical Slice)	Funcionalidade completa (entrada + caso de uso) em uma única pasta.
Microfrontend	Parte da interface publicada de forma independente e composta por um shell.
Porta / Adaptador	Interface definida pela aplicação / implementação concreta dessa interface.
Réplica local	Cópia mínima de dados de outro serviço, mantida por eventos.
Serverless	Modelo de execução sem gestão de servidores (AWS Lambda), cobrado por uso.
Taxa de entrega	Valor fixo de 790 centavos (R$ 7,90) somado ao subtotal do pedido.